

CS330 Operating Systems and Lab.

File Systems 2

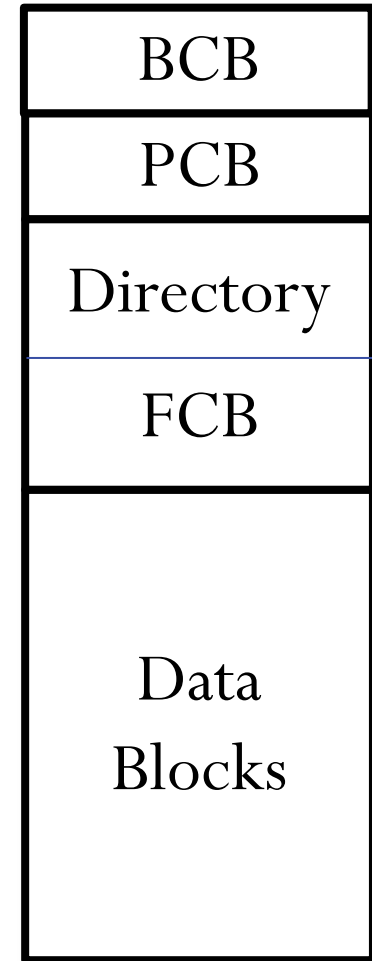
Spring 2026

KAIST

Instructor : Insik Shin

File-System Implementation

- **Boot Control Block (BCB)** contains info needed by system to boot OS from that partition
 - Needed if partition contains OS, usually first block of partition
- **Partition Control Block (PCB)** contains partition details (FS metadata)
 - Total # of blocks, # of free blocks, block size, free block pointers
- **Directory structure** organizes the files
 - Names and inode numbers, master file table
- Per-file **File Control Block (FCB)** contains many details about the file (file metadata)
 - Inode number, permissions, size, dates



Naming

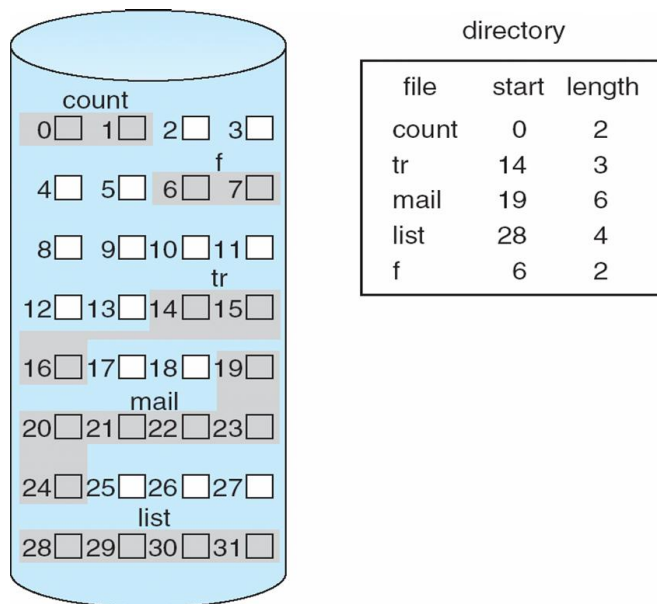
- How many disk accesses to resolve

“/users/i/insik.shin/thumbnail.jpg” (1KB)?

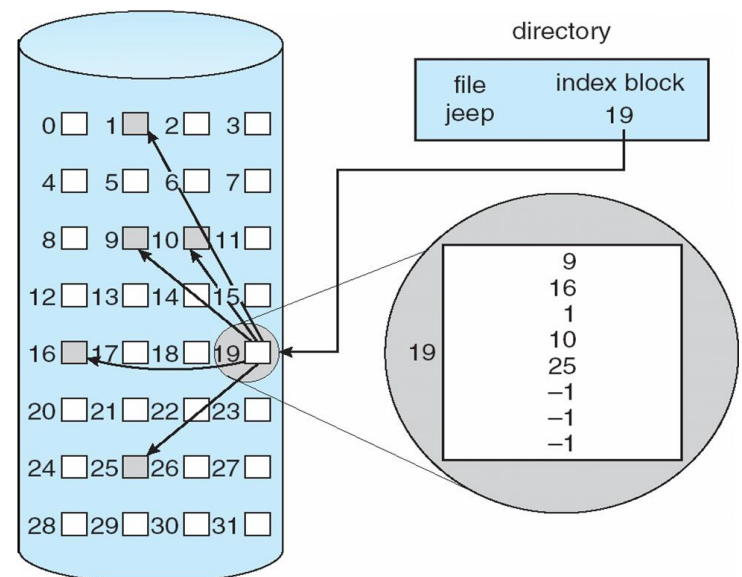
Allocation Methods

- Allocation methods for how to allocate disk blocks to files:

Contiguous allocation

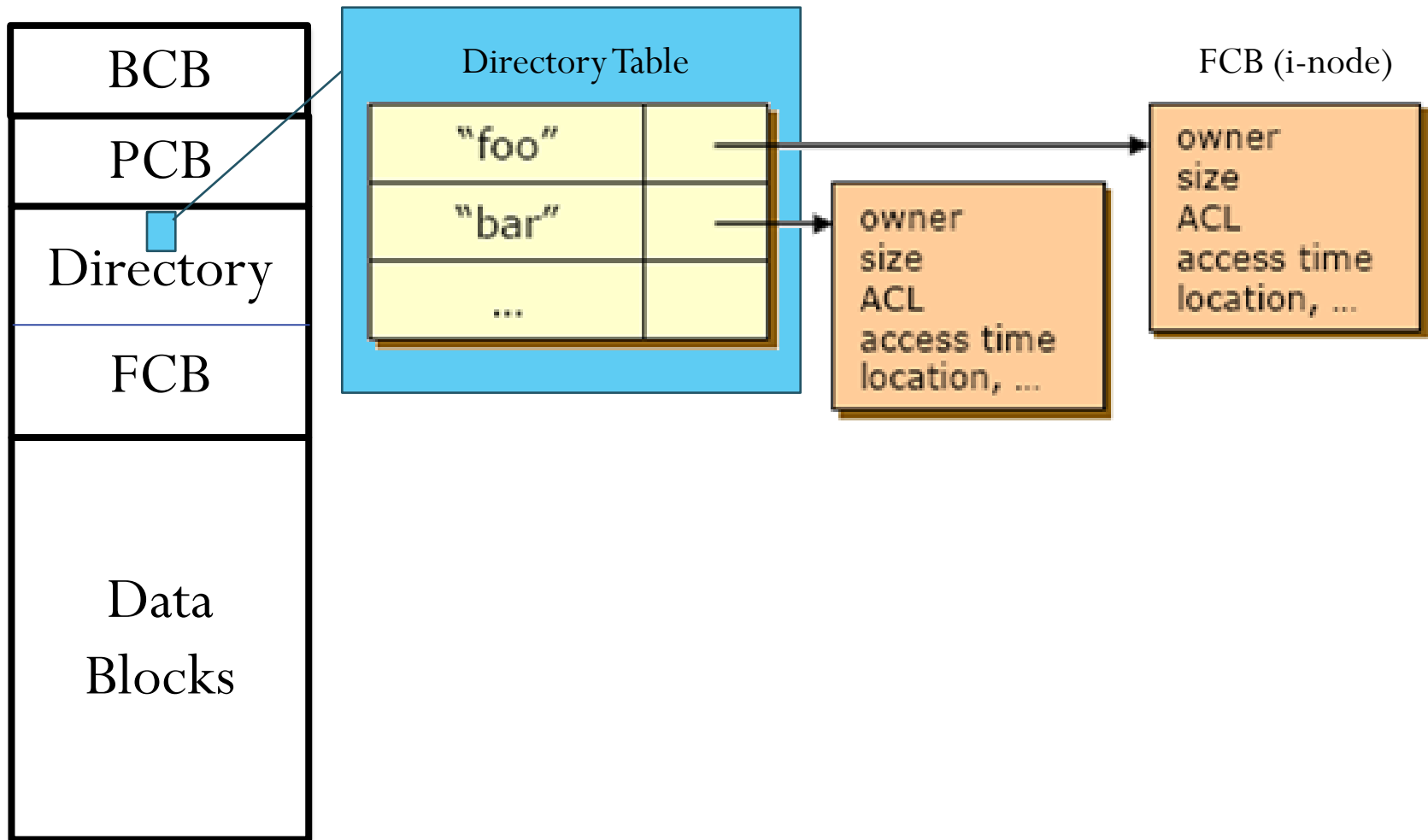


Indexed allocation

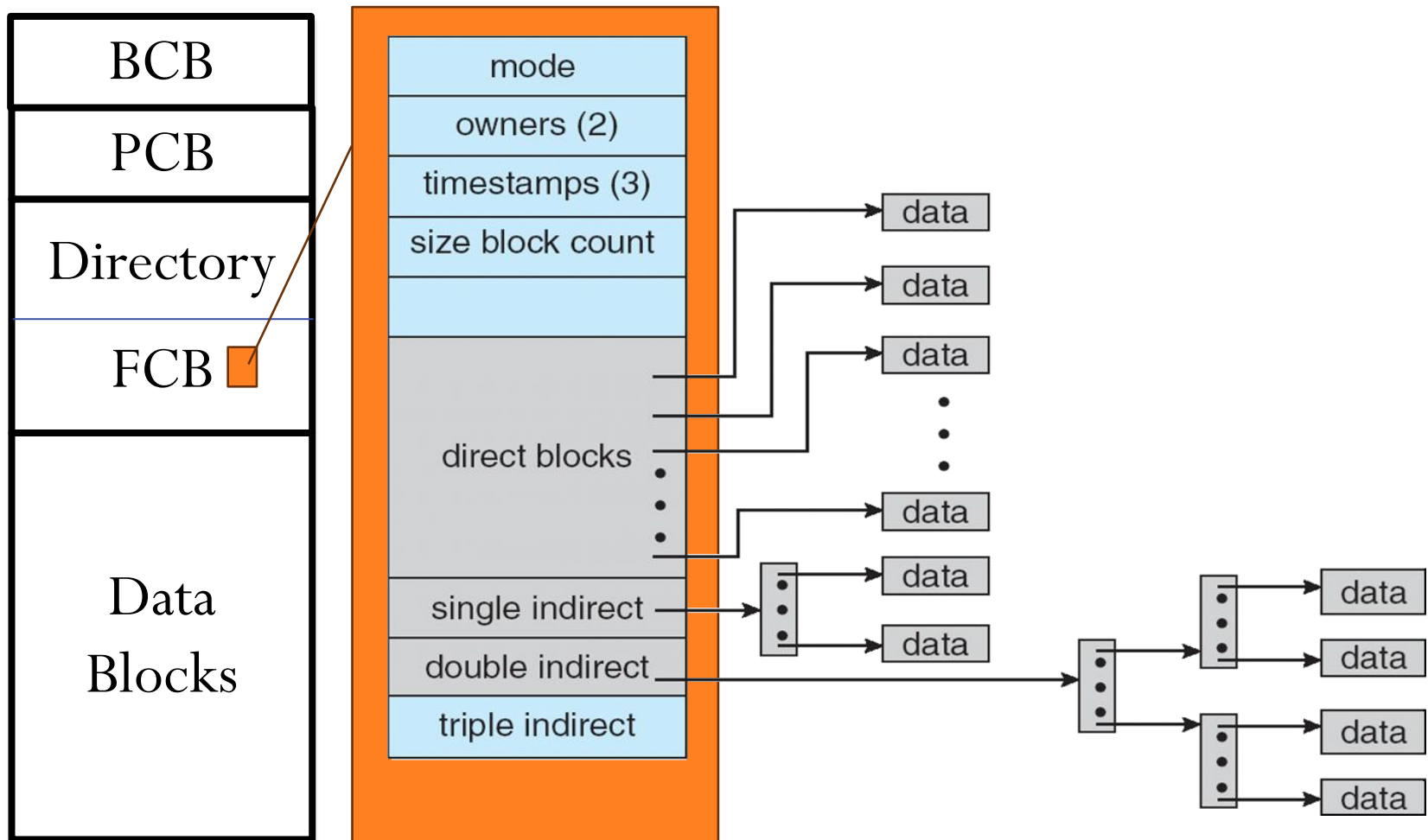


Directory Structure

- Metadata are stored in the file control block (i.e., i-node)

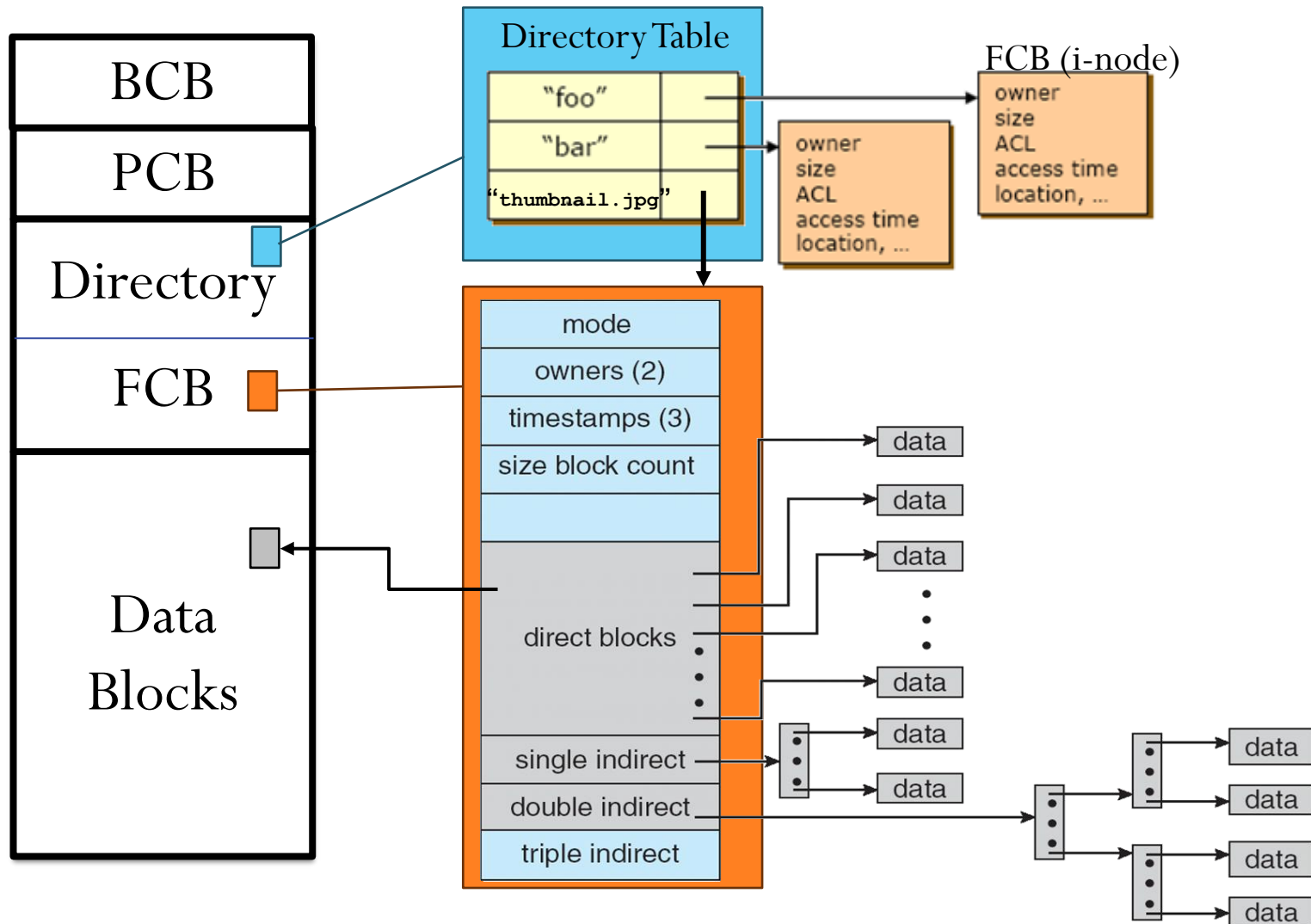


Combined Scheme: UNIX UFS (4K bytes per block)



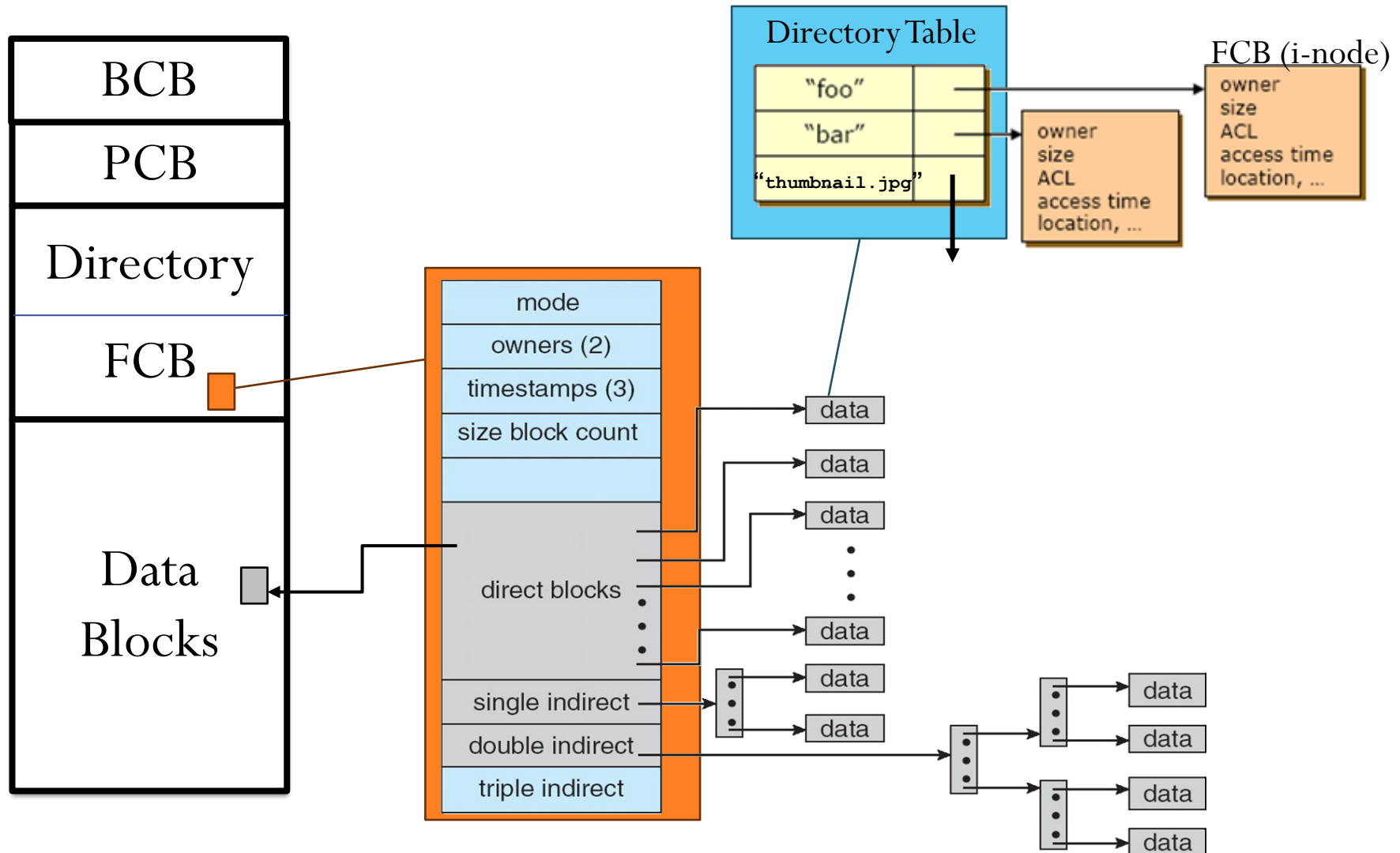
Directory Structure

/users/i/insik.shin/thumbnail.jpg



Directory Structure

/users/i/insik.shin/thumbnail.jpg

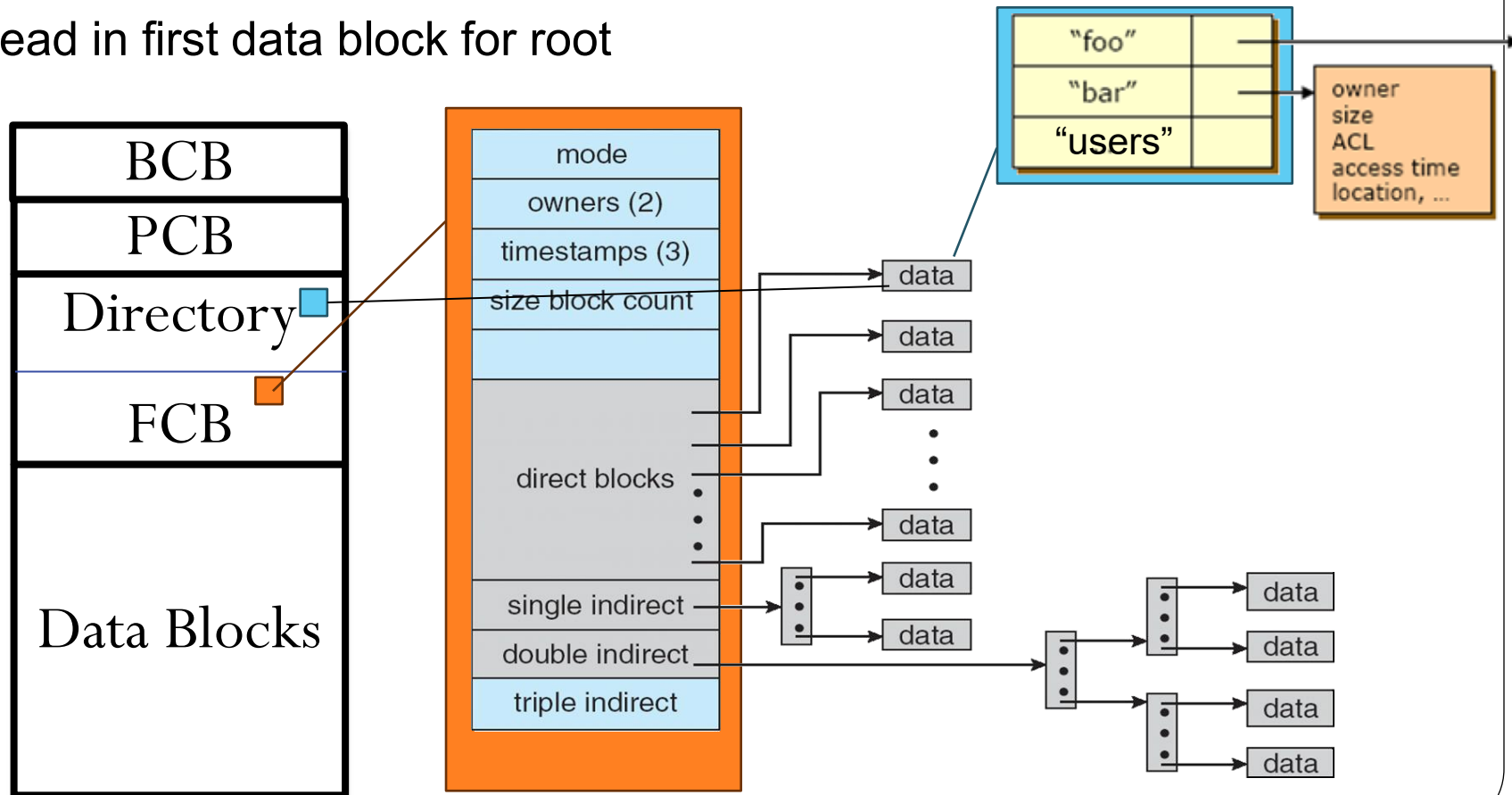


Naming

- How many disk accesses to resolve

`“/users/i/insik.shin/thumbnail.jpg” (1KB)?`

- 1) Read in i-node for root (fixed spot on disk)
- 2) Read in first data block for root

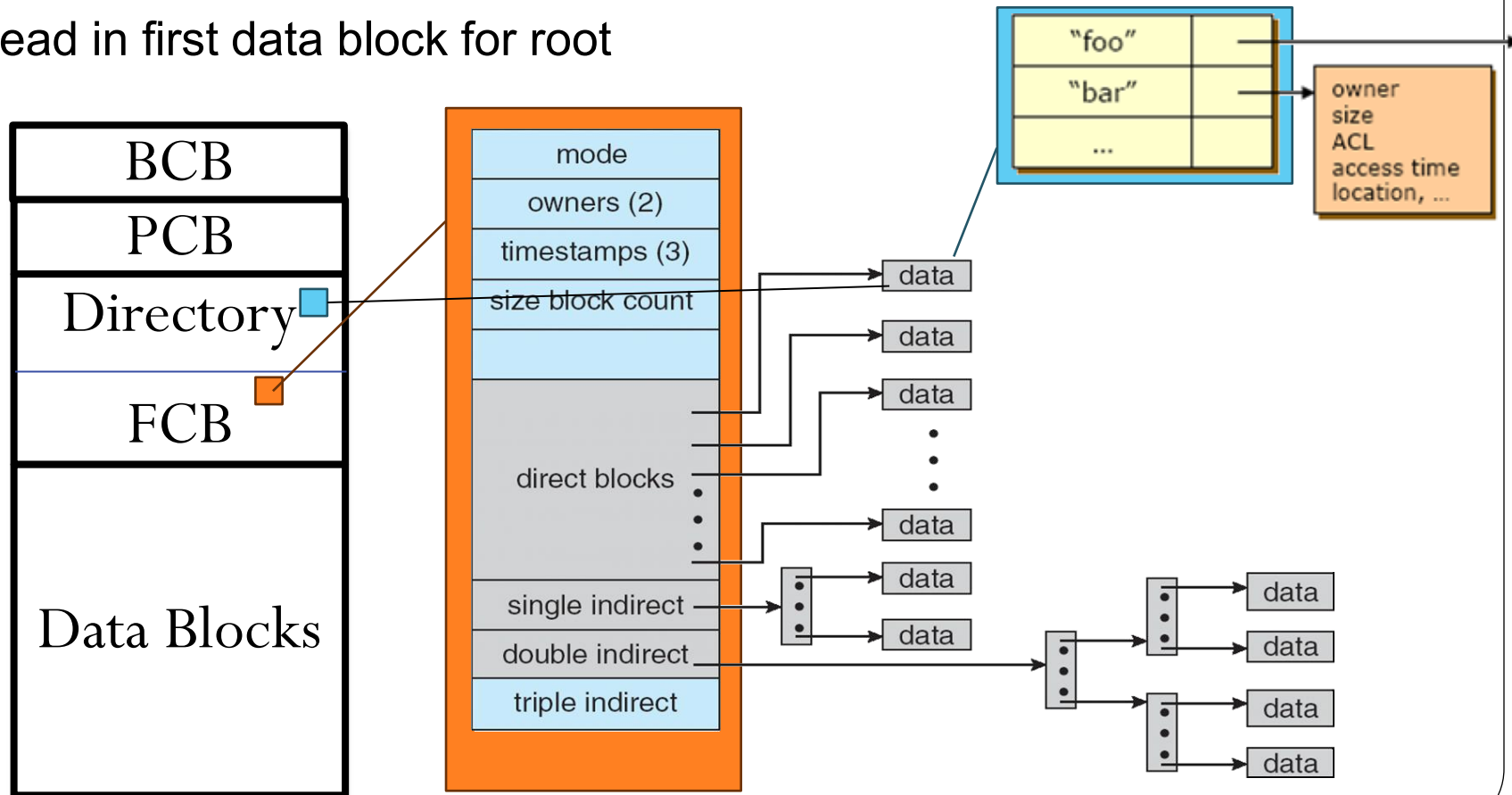


Naming

- How many disk accesses to resolve

“/users/i/insik.shin/thumbnail.jpg” (1KB)?

- 1) Read in i-node for root (fixed spot on disk)
- 2) Read in first data block for root



Naming

- How many disk accesses to resolve

“/users/i/insik.shin/thumbnail.jpg” (1KB)?

- 1) Read in i-node for root (fixed spot on disk)
- 2) Read in first data block for root
- 3) Read in i-node for “users”
- 4) Read in first data block for “users”; search for “i”
- 5) Read in i-node for “i”
- 6) Read in first data block for “i”; search for “insik.shin”
- 7) Read in i-node for “insik.shin”
- 8) Read in first data block for “insik.shin”; search for “thumbnail.jpg”
- 9) Read in i-node for “thumbnail.jpg”
- 10) Read in first data block for “thumbnail.jpg”

based on OSDI 2010 Paper

Finding a Needle in a Haystack: Facebook's Photo Storage



Facebook

- Social network service website
- How many photos are uploaded/stored/serviced at Facebook ?



Photos @ Facebook

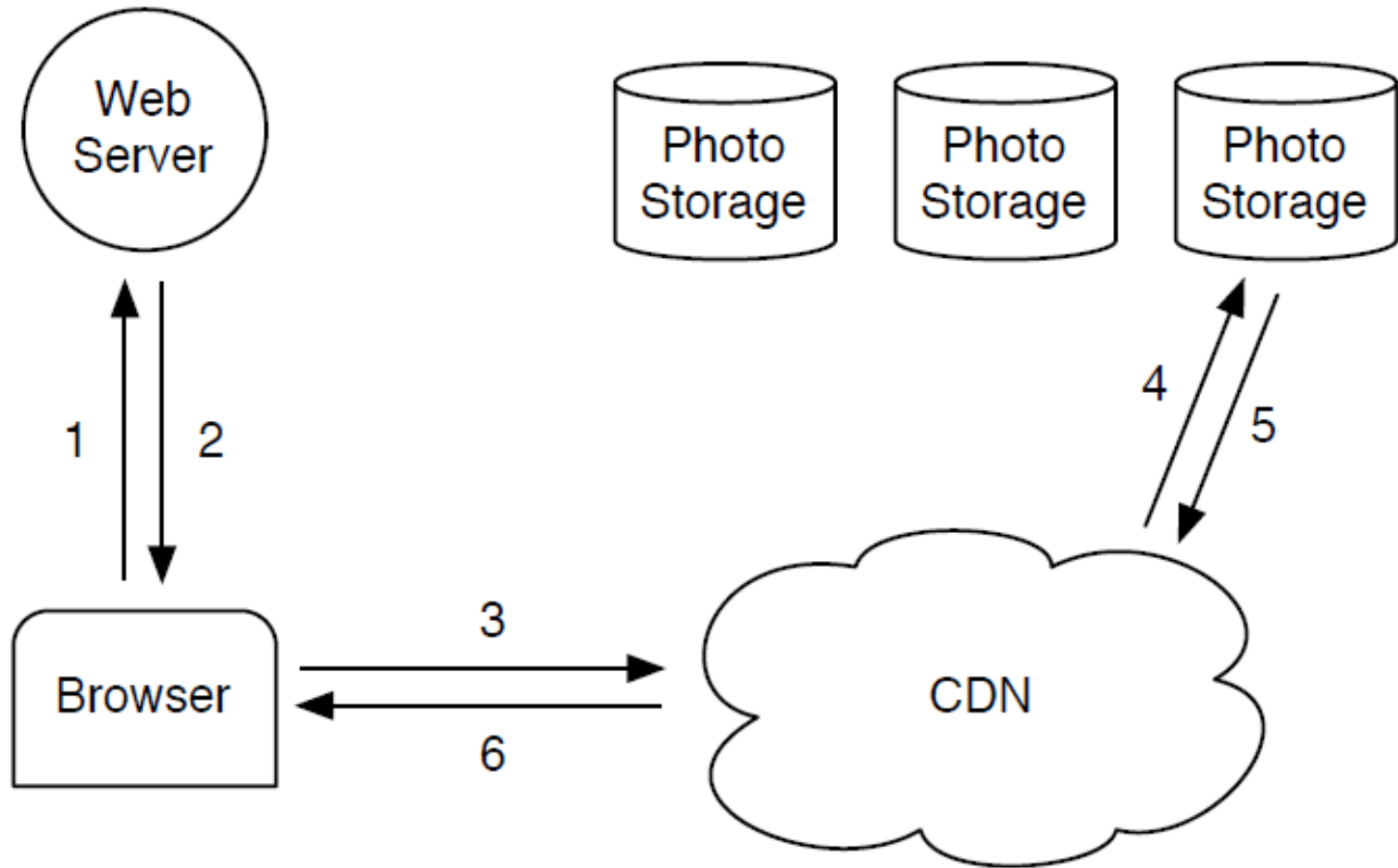
- How many photos & images @ Facebook ?
 - 65 billion photos & 260 billion images
 - 20 petabytes (PB)
- Upload rate
 - users upload one billion new images each week
 - 60 TB of data
- Serving rate
 - Facebook serves over one million images per second at peak

How to design a “good” photo serving system for Facebook ?

Main Goals for Photo Serving Method

- Four main goals for photo serving method:
 - High throughput and low latency
 - Provide a good user experience
 - Fault-tolerant
 - Handle server crashes and hard drive failures
 - FB's data of 20PB required 20K hard disks of 10TB
- About 20 disks can fail everyday (typical lifetime of a hard disk is 3~5 years)
 - Cost-effective
 - Save money over traditional approaches (reduce reliance on CDNs!)
 - Simplicity
 - Make it easy to implement and maintain

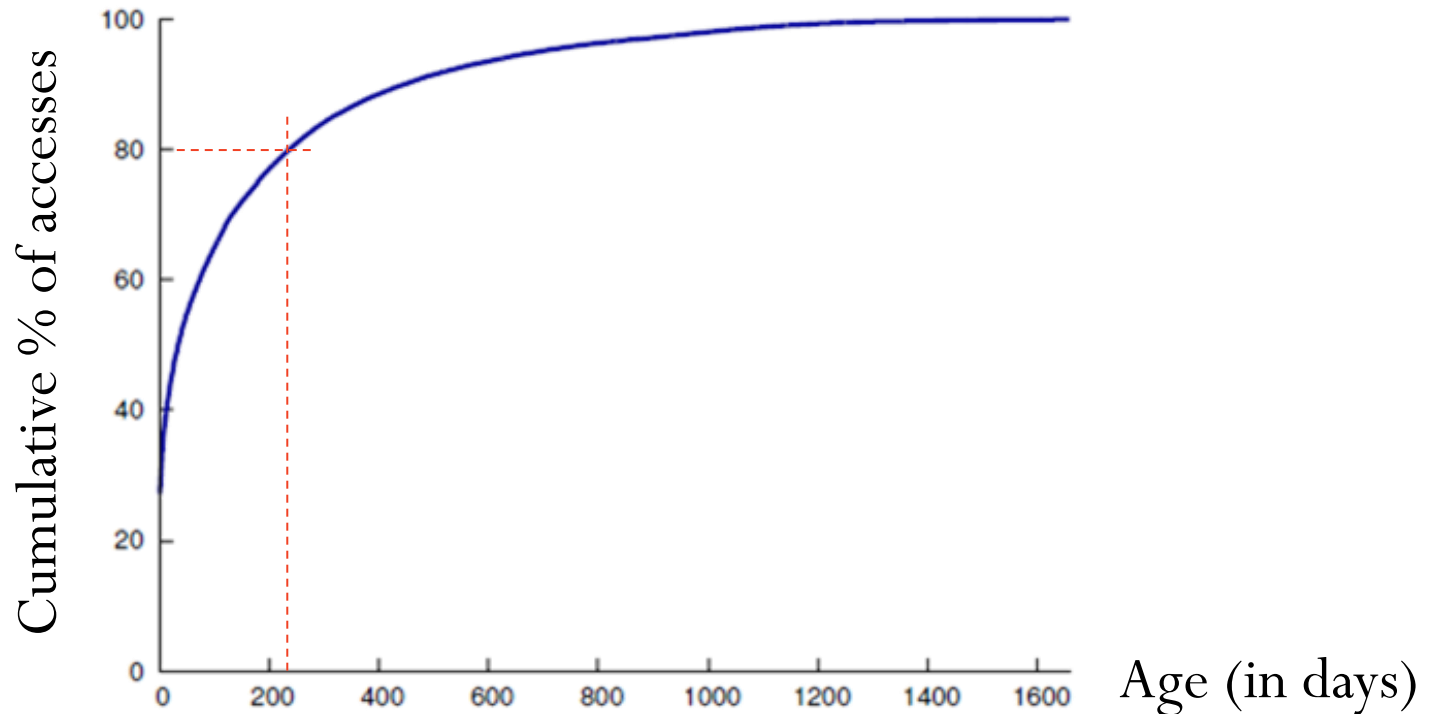
Typical Design



A **content delivery network** or **content distribution network (CDN)** is a large distributed system of servers deployed in multiple data centers across the Internet.

Background

- Two types of workloads for image serving
 - Profile pictures – heavy access, smaller size
 - Photo albums – intermittent access, higher at beginning, decreasing over time (long tail)

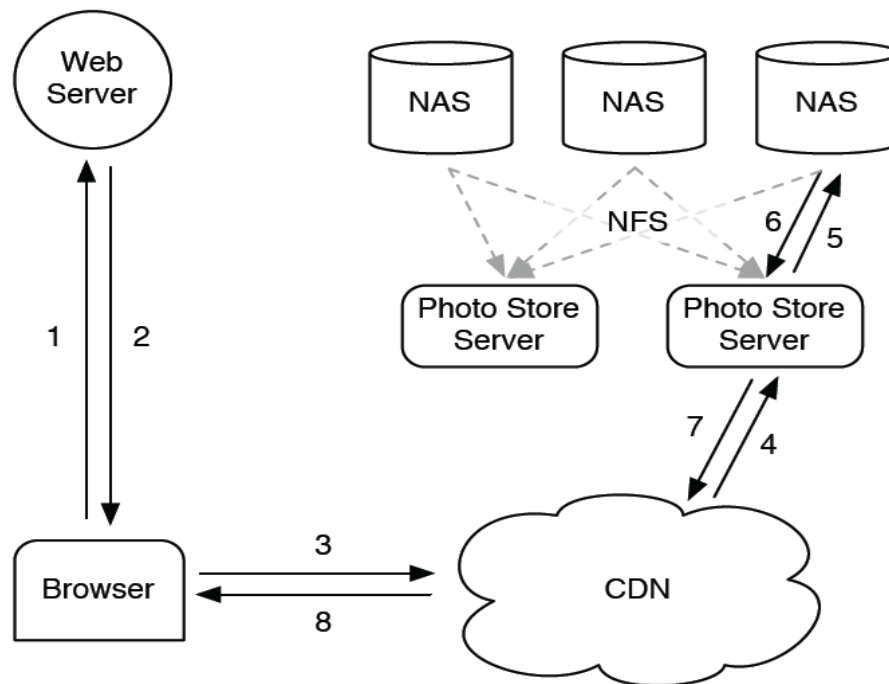


Facebook vs. Typical Websites

- Typical websites
 - Small working set
 - Infrequent access of old content
 - ~99% CDN hit rate
- Facebook
 - Large working set
 - Frequent access of old content
 - 80% CDN hit rate

Facebook's Old Design

- The old photo infrastructure consisted of several tiers:
 - **Upload tier** receives users' photo uploads, scales the original images and saves them on the NFS storage tier.
 - **Photo serving tier** receives HTTP requests for photo images and serves them from the NFS storage tier.
 - **NFS storage tier** built on top of commercial storage appliances



NFS based Design

- Metadata bottleneck
 - Each image stored as a file
 - Large metadata size severely limits the metadata hit ratio (in the cache)
- Image read performance
 - ~10 iops / image read (large directories – thousands of files)
 - ~3 iops / image read (smaller directories – hundreds of files)
 - read the directory metadata into memory
 - load the inode into memory
 - read the file contents
 - ~2.5 iops / image read (file handle cache)
 - Cache the filename-to-file-handle mapping
 - Custom system call: `open_by_filehandle`
 - A minor improvement (a low cache hit ratio)

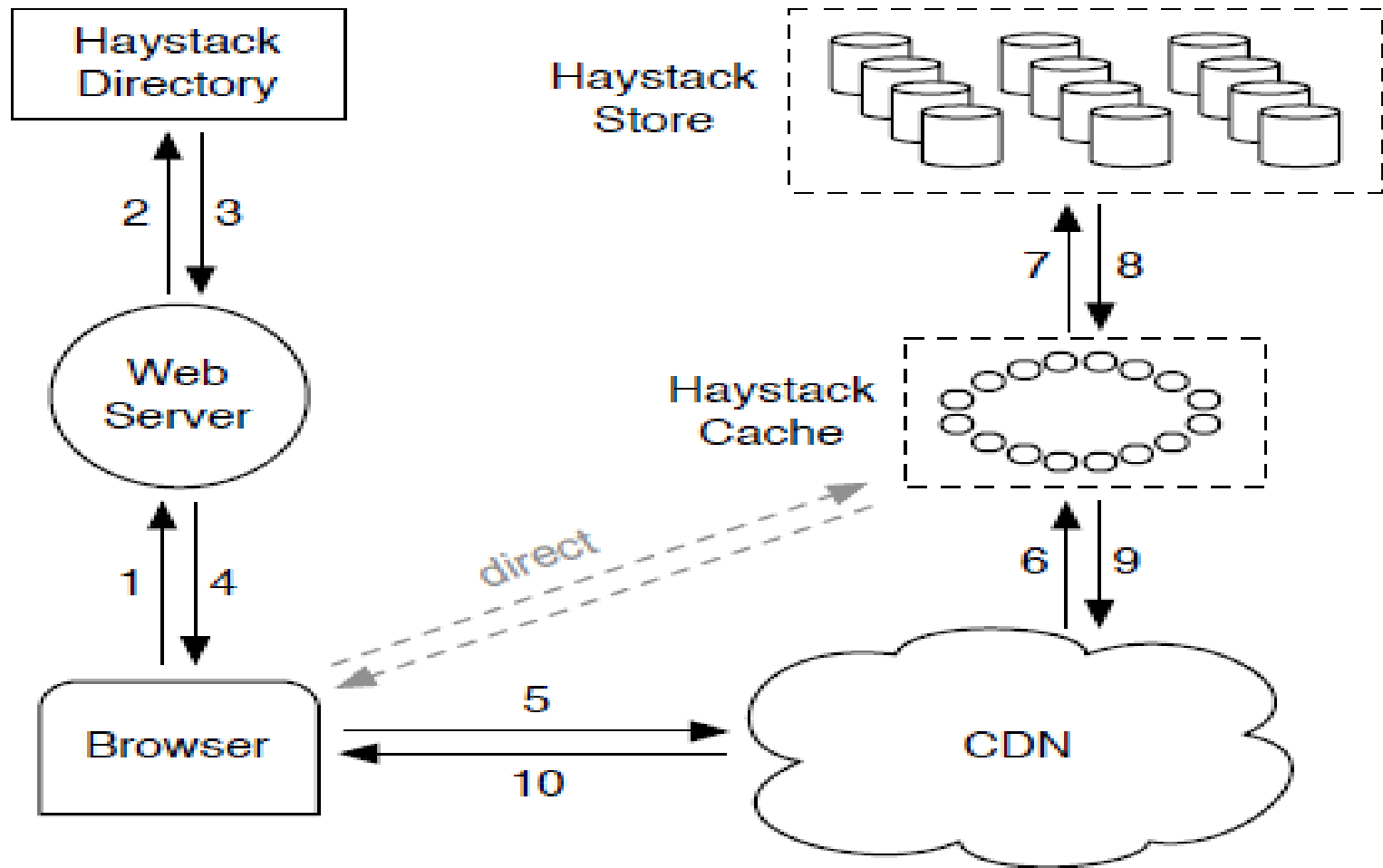
Haystack

- Haystack
 - Addresses the critical bottleneck in NFS-based approach
 - Aims to limit the number of operations to only the ones necessary
 - Reduces the memory used for filesystem metadata
 - Stores multiple photos in a single file
 - Two kinds of metadata
 - Application metadata
 - Information needed to construct a URL
 - Filesystem metadata
 - Identify the data necessary for a host to retrieve the photos

Keystone of the Haystack design

- Aims to retrieve the filename, offset, and size of a particular photo without any disk operations
 - On a disk, a single very large file (100GB) is saved as 'hay/haystack_<logical volume id>'

Haystack



Step-through of Operation

- When a user visits a page,
 - Web server receives the request and uses Haystack Directory to construct URL for each photo
 - `http://<CDN>/<Cache>/<Machine id>/<Logical volume, Photo>`
 - From which CDN to request the photo
 - This portion may be omitted if the photo is available directly from the Cache
 - If CDN is unsuccessful, contacts the Cache

Haystack

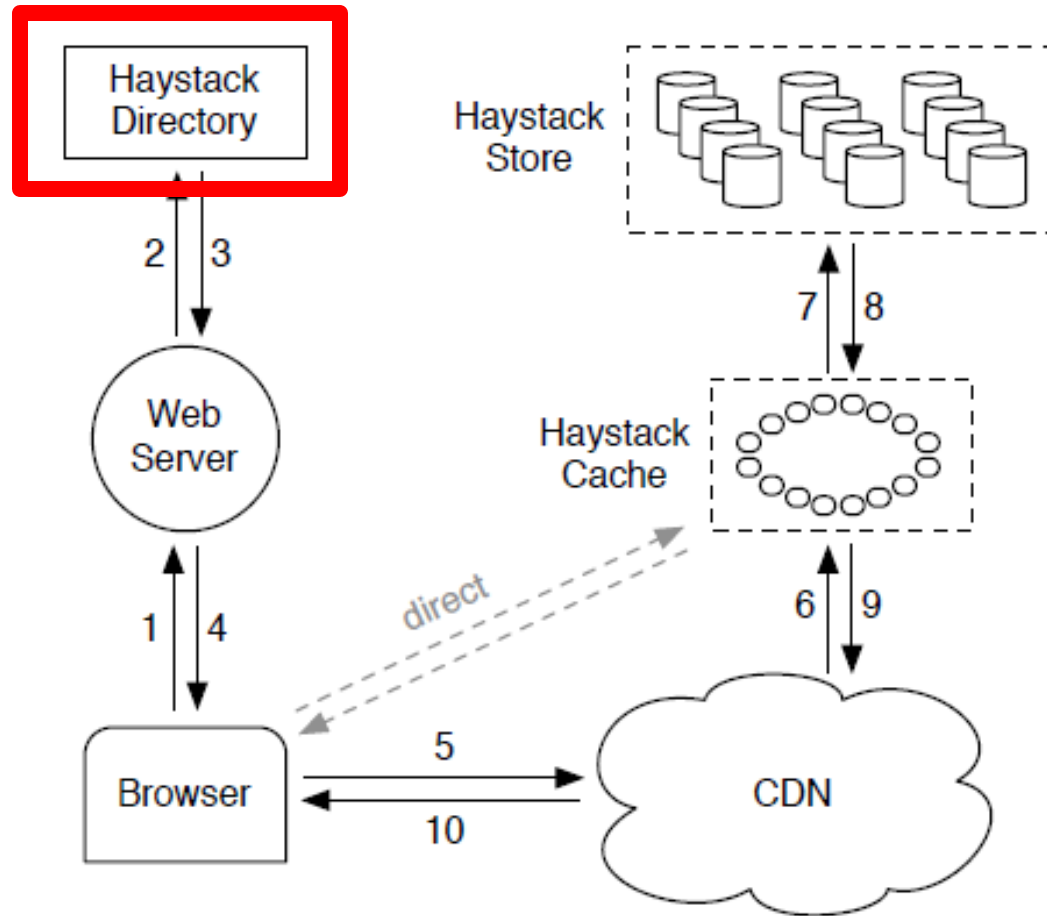


Figure 3: Serving a photo

Haystack Directory

- Four main functions...
 - Determines whether a photo request should be handled by the CDN or by the Haystack Cache
 - Provides a mapping from logical volumes to physical volumes
 - Load balances writes across logical volumes
 - Identifies logical volumes that are read-only
 - Operational reasons
 - Reached storage capacity

Haystack

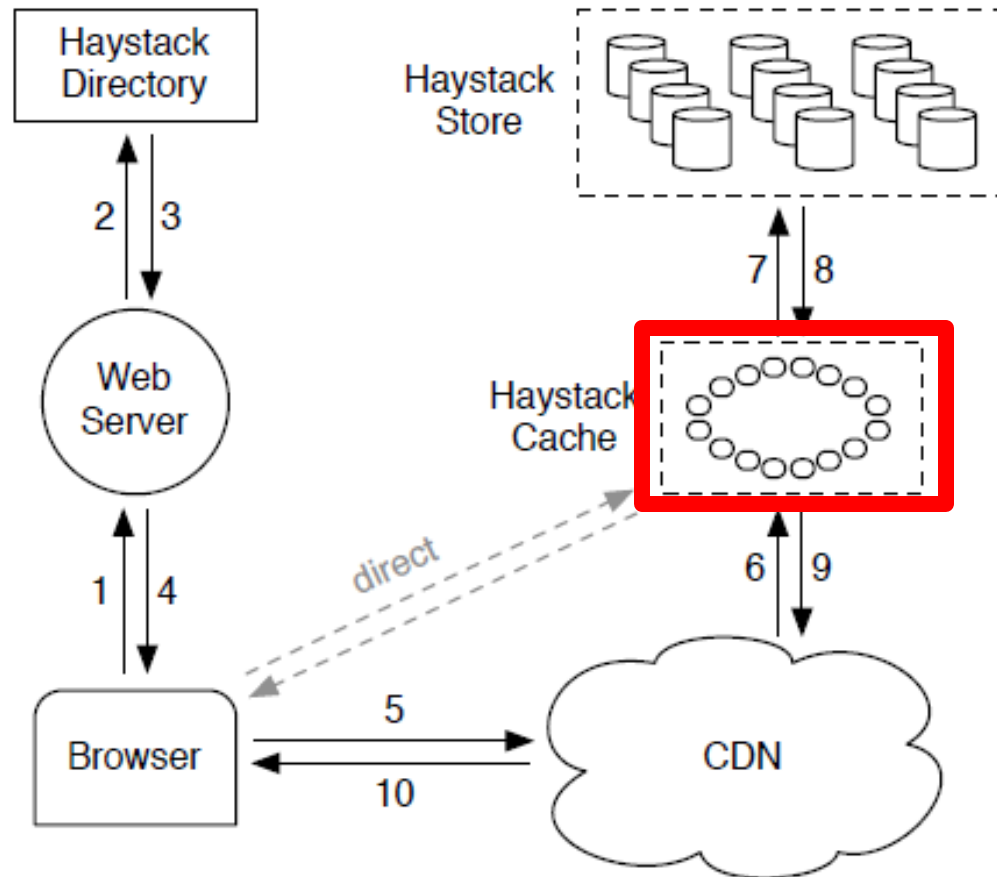
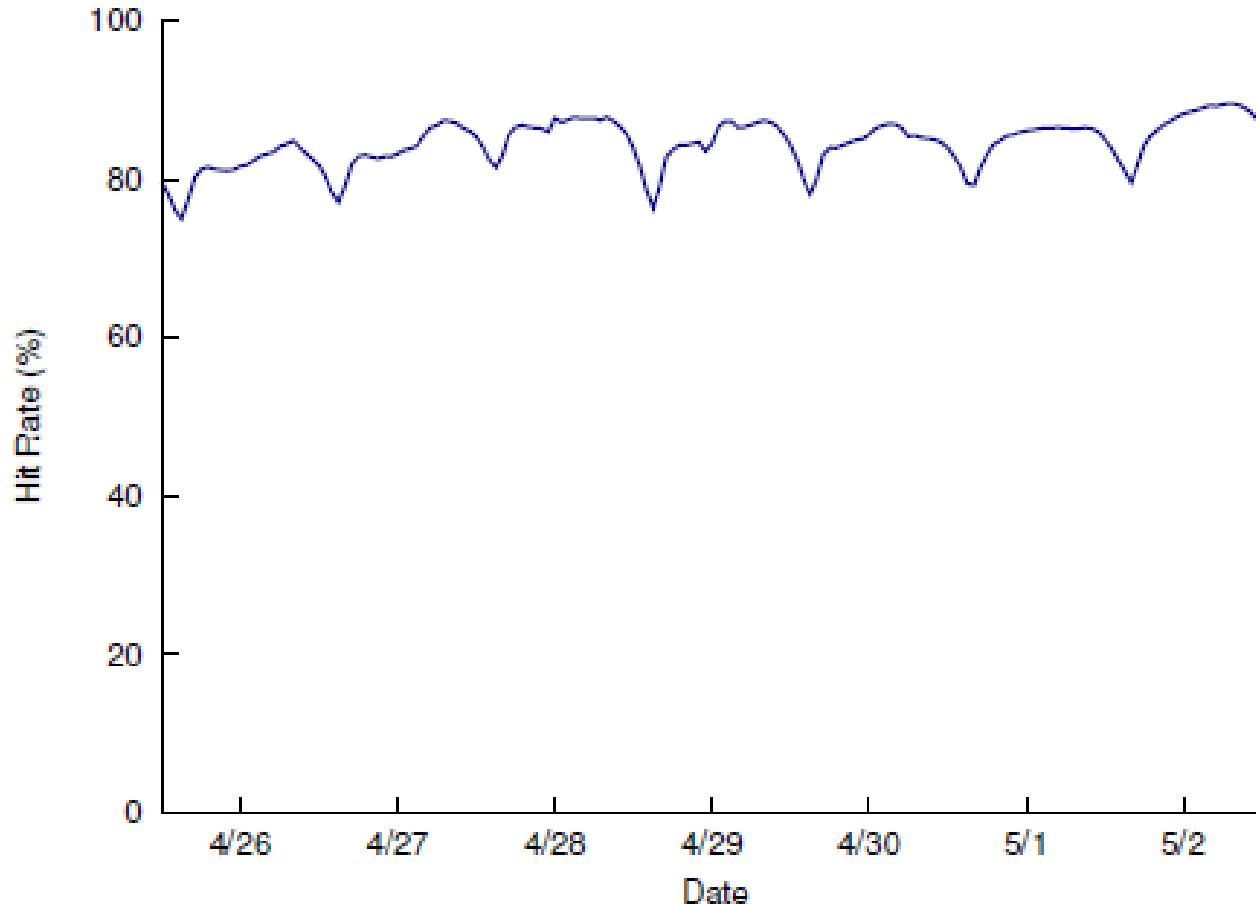


Figure 3: Serving a photo

Haystack Cache

- Is a distributed hash table & uses photo's id to locate cached data
- Receives HTTP requests from CDNs and browsers
 - If photo is in Cache, return the photo
 - If photo is not in Cache, fetches photo from the Haystack Store and returns the photo
- Adds a photo to Cache if two conditions are met...
 - The request comes directly from a browser, not the CDN
 - The photo is fetched from a write-enabled Store machine

Cache Hit Rate



Cache hit rate for images that might be potentially stored in the Haystack Cache

Haystack

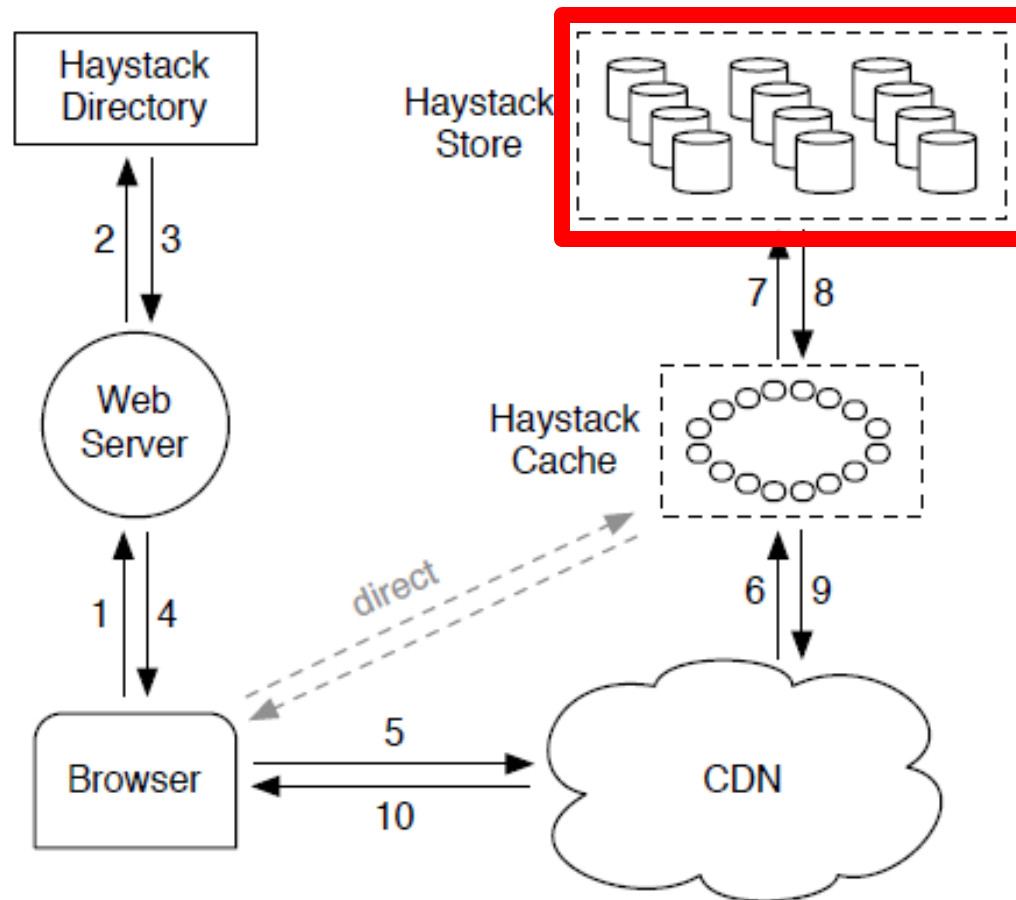
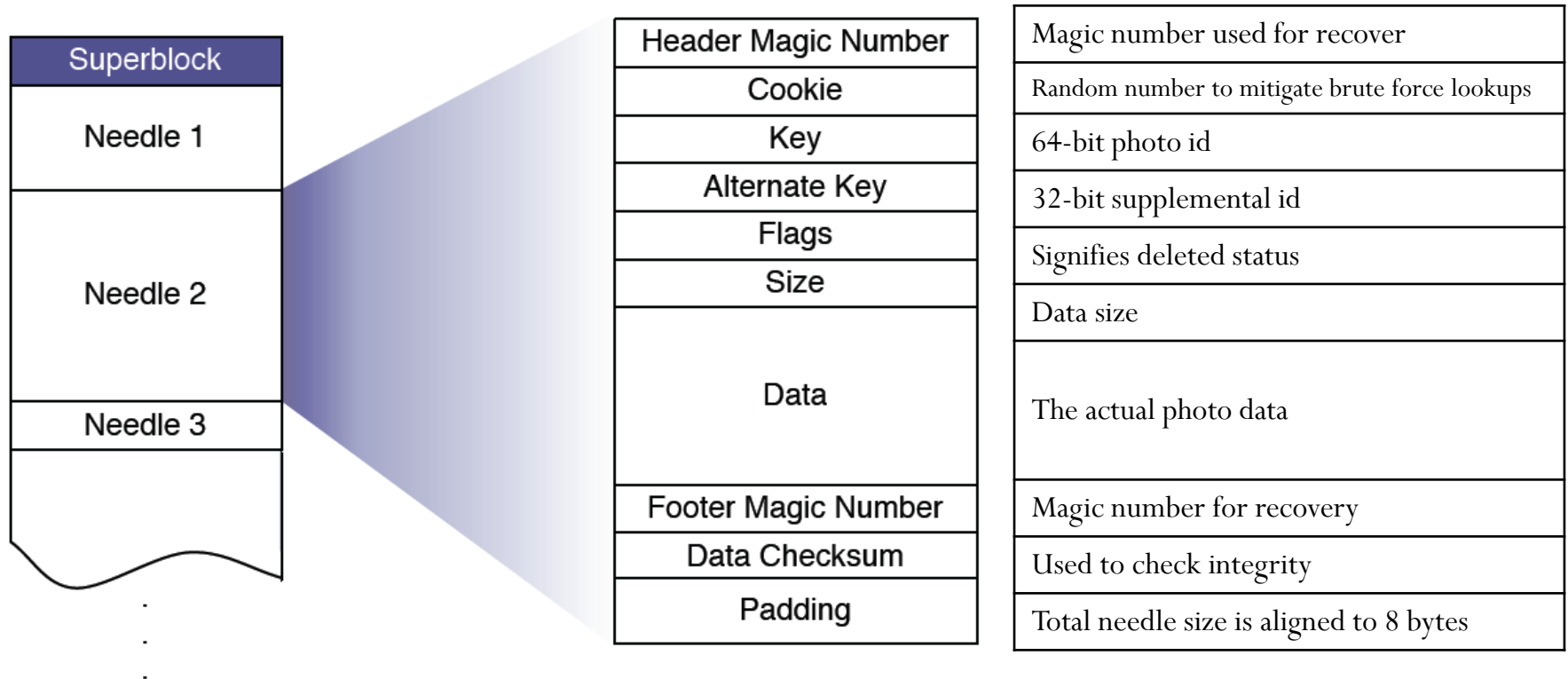


Figure 3: Serving a photo

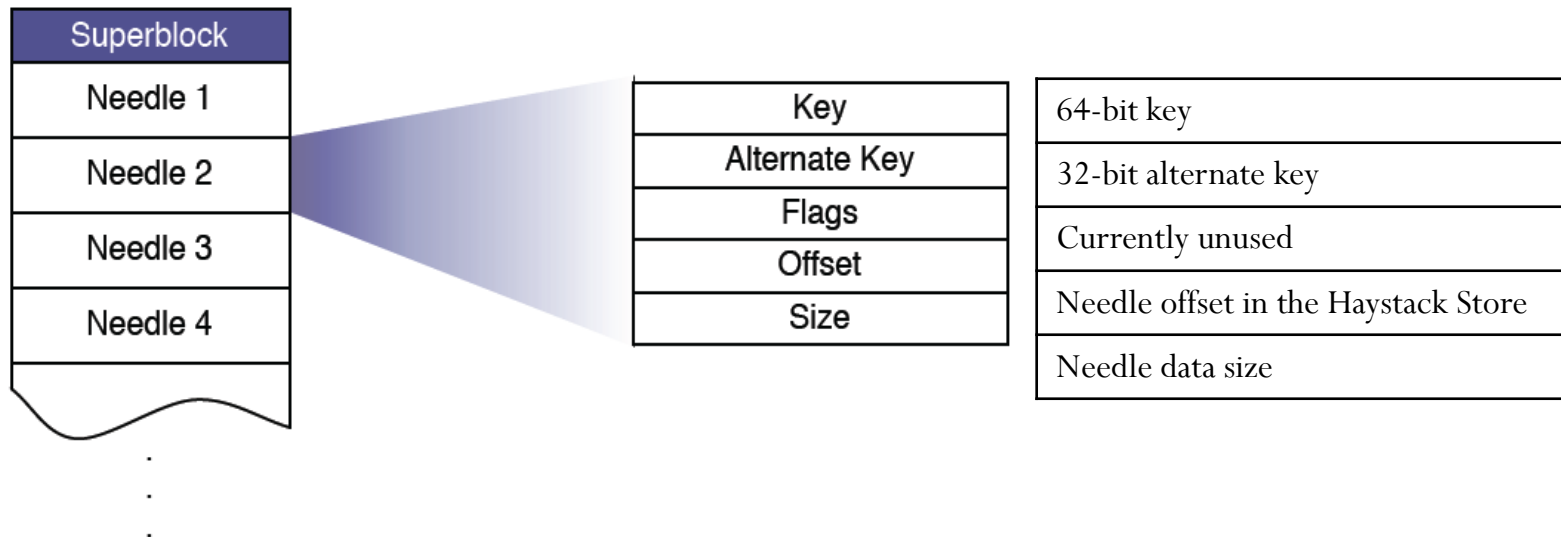
Layout of Haystack Store file

- A *Haystack Store* file is a logical storage, and a *needle* is for a single image
- Main purpose: to reduce file system-level metadata (namespace directory and inode) (how ?)



Haystack Index File

- The index file provides the minimal metadata required to locate a particular needle in the store
 - Main purpose: allow quick loading of the needle metadata into memory without traversing the larger Haystack store file
 - Index is usually less than 1% the size of the store file



Haystack - Summary

- Simplified metadata
 - No directory structures/file names
 - 64-bit ID
 - Results in easier lookups
- Reduced disk I/O
 - Optimized for random reads (~1 I/O per object read)
 - 1 MB of metadata for every 1GB of usable storage
 - 10 TB/node results in 10 GB of metadata
 - This amount is easily cacheable!
- 8,500 LOC (C++)
 - 2 engineers 4 months from inception to initial development